Android Studio Setup Guide

Build a 2D Pixel-Art Bullet-Heaven Survival Game with Java and Android Canvas

Goal	Set up Android Studio, create a Java project, launch an emulator, and run a simple game screen.
Recommended approach	Android Canvas / SurfaceView for a lightweight custom 2D game.
Laptop context	GTX 1650 laptop: suitable for 2D development; emulator performance also depends on CPU virtualization
First milestone	Run the app and display a player placeholder before importing large art assets.

Important: Start with a small working project. Add the map, sprites, enemies, and combat gradually so you can isolate bugs and avoid filling your SSD with unnecessary downloads.

1. Install Android Studio

1. Open the official download page: developer.android.com/studio.

2. Download the Windows installer and run it.

3. In the setup wizard, keep Android Studio and Android SDK selected. Install the Android Virtual Device component if offered.

4. Launch Android Studio and allow the setup wizard to download the required SDK components.

5. If your SSD space is limited, choose a drive with more free space for SDKs, projects, and emulator images. For example:

```
D:\Android\Android Studio
D:\Android\Sdk
D:\Projects\BulletHeaven
```

These are example paths, not required names. You can keep Android Studio on one drive and put the SDK and projects on another if the installer/settings allow it.

2. Create the Java game project

1. From the welcome screen, choose **New Project**.

2. Select **Empty Views Activity** (not Empty Activity if that template means Jetpack Compose in your Android Studio version).

3. Set the app name to **ArcaneSurvivor** or another name you prefer.

4. Choose **Java** as the language.

5. Choose a minimum SDK such as **API 26** unless you have a reason to support older devices.

6. Click Finish and wait for Gradle sync to complete.

The initial project is just a normal Android app. You will add a custom drawing View or SurfaceView to turn it into a game.

3. Choose and create an emulator

Recommended starting point: use the built-in Android Emulator with a Pixel phone profile. A Pixel 7a profile is a good starting choice; choose a similar available Pixel profile if it is not listed. Use a recommended stable Android system image, such as API 35 or whichever stable API Android Studio recommends.

To create it:

1. On the welcome screen, open **More Actions** → **Virtual Device Manager**. In an open project, use **View** → **Tool Windows** → **Device Manager**.

2. Click **+** or **Create Virtual Device**.

3. Choose **Phone**, then select a Pixel device profile.

4. Select a recommended system image. If it is not installed, click Download and wait for it to finish.

5. Finish the wizard, then click the play/launch icon beside the new virtual device.

Suggested first settings

Setting	Starting choice
Graphics	Automatic / Hardware
RAM	2–3 GB if configurable
CPU cores	2–4 if configurable
Resolution	Default device profile

Setting	Starting choice
Boot	Quick Boot / default

The emulator is not powered only by the GTX 1650. CPU virtualization support is important. If the emulator is slow or refuses to start, check that virtualization is enabled in BIOS/UEFI and follow the official acceleration guide: Android Emulator acceleration.

4. Run the app

1. Wait for Gradle sync to finish without errors.

2. Start the virtual device and wait until Android has booted to its home screen.

3. In Android Studio's device selector, choose the running emulator.

4. Click the green **Run** ■ button.

5. Android Studio builds and installs your app. The app should open inside the emulator.

If the emulator is too slow, close other heavy programs, use a smaller phone profile, avoid running multiple emulators, and verify virtualization. For a 2D game, you do not need an expensive or high-resolution virtual phone.

5. Add your game files

A simple beginner-friendly structure is:

```
app/src/main/java/your/package/
    MainActivity.java
    GameView.java

app/src/main/assets/
    maps/
    characters/
    enemies/
    projectiles/
    effects/
    audio/
```

Keep images in assets so you can replace your pixel art without rewriting the game logic. You can begin with a colored circle instead of a sprite.

6. What to build first

Milestone 1 — Display: Show a background and a player placeholder.

Milestone 2 — Controls: Make the player move with touch input or a virtual joystick.

Milestone 3 — Camera: Keep the player near the screen center while the world scrolls.

Milestone 4 — Map: Draw a small tile map and add obstacle collisions.

Milestone 5 — Enemies: Spawn enemies and make them follow the player.

Milestone 6 — Combat: Add automatic attacks, projectiles, hit detection, and health.

Milestone 7 — Progression: Add XP gems, level-ups, and simple upgrades.

Milestone 8 — Polish: Add sprite-sheet animation, sound, pause, and settings.

7. Test on a real Android phone

1. On the phone, open Settings → About phone and tap Build number several times to enable Developer options (the exact menu can vary by manufacturer).

2. Open Developer options and enable USB debugging.

3. Connect the phone to the laptop with a USB data cable and accept the authorization prompt on the phone.

4. Select the phone from Android Studio's device selector and click Run.

8. Useful official links

• **Android Studio download and install:** https://developer.android.com/studio

• **Create and manage virtual devices:** https://developer.android.com/studio/run/managing-avds

• **Emulator acceleration:** https://developer.android.com/studio/run/emulator-acceleration

- **Run apps on a hardware device:** https://developer.android.com/studio/run/device
- **Android graphics and drawing overview:** https://developer.android.com/develop/ui/views/graphics

Final recommendation

Install Android Studio, create the Java Empty Views Activity project, launch one emulator, and get a basic screen running before downloading large asset packs. The first meaningful goal is a moving player marker; then add camera scrolling, maps, enemies, and combat in that order.